Experiment 2

Linear Regression

Aim:

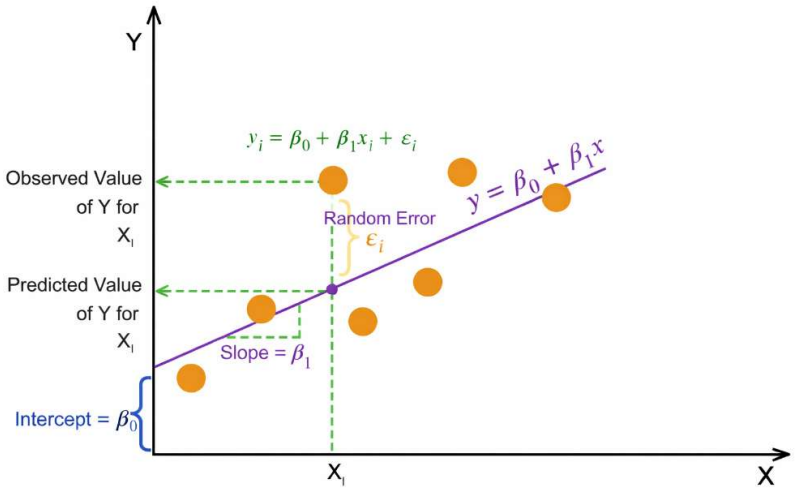
To implement and visualize simple linear regression and multiple linear regression, and to evaluate model performance using standard regression metrics.

Theory:

Linear Regression is a supervised machine learning algorithm used to model the relationship between a dependent (target) variable and one or more independent (feature) variables by fitting a linear equation to observed data. The objective is to predict continuous numerical outcomes.

Simple Linear Regression: involves a single independent variable and is mathematically expressed as:

$$Y = \beta_0 + \beta_1 X + \epsilon$$



Multiple Linear Regression: involves two or more independent variables and is expressed as:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n + \epsilon$$

Where,

- Y: Dependent variable (Target)
- X: Independent variable(s) (Features)
- β_0: Intercept

- $\beta_1, \beta_2, \ldots, \beta_n$: Regression coefficients
- ϵ: Error term (residual)

Method of Least Squares

The coefficients of the regression model are estimated using the **Least Squares Method**, which minimizes the **Residual Sum of Squares (RSS)**—the sum of squared differences between observed and predicted values.

Assumptions of Linear Regression

1. **Linearity** – The relationship between independent and dependent variables is linear.
2. **Independence** – Observations are independent of each other.
3. **Homoscedasticity** – Constant variance of residuals across all values of X.
4. **Normality** – Residuals are normally distributed.

Merits of Linear Regression

- **Simplicity and Interpretability:** Linear Regression is easy to understand and implement. Model coefficients have clear statistical meaning, indicating the magnitude and direction of feature influence on the target variable.
- **Effective for Linearly Related Data:** Performs well when the true relationship between features and target is approximately linear and assumptions are reasonably satisfied.

Demerits of Linear Regression

- **Linearity Assumption:** The model cannot capture non-linear relationships unless manual feature transformations (e.g., polynomial terms) are applied.
- **Sensitivity to Outliers:** Least Squares estimation squares residuals, giving excessive weight to outliers, which can significantly distort the model.
- **Multicollinearity Issues:** High correlation among independent variables leads to unstable coefficient estimates and reduced interpretability in Multiple Linear Regression.

Code Block 1: Multiple Linear Regression; Importing Libraries

Input

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

Output

Code Block 2: Reading Data

Input

```
data = pd.read_csv('/kaggle/input/car-data/car data.csv')
```

Output

Code Block 3: Reading Data

Input

```
rows_to_show = [31, 144, 42, 114, 89]
data.loc[rows_to_show]
```

Output

Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner	
31	ritz	2011	2.35	4.89	54200	Petrol	Dealer	Manual	0
144	Bajaj Pulsar NS 200	2014	0.60	0.99	25000	Petrol	Individual	Manual	0
42	sx4	2008	1.95	7.15	58000	Petrol	Dealer	Manual	0
114	Royal Enfield Classic 350	2015	1.15	1.47	17000	Petrol	Individual	Manual	0
89	etios g	2014	4.75	6.76	40000	Petrol	Dealer	Manual	0

Code Block 4: Reading Data

Input

data.head()

Output

Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner	
0	ritz	2014	3.35	5.59	27000	Petrol	Dealer	Manual	0
1	sx4	2013	4.75	9.54	43000	Diesel	Dealer	Manual	0
2	ciaz	2017	7.25	9.85	6900	Petrol	Dealer	Manual	0
3	wagon r	2011	2.85	4.15	5200	Petrol	Dealer	Manual	0
4	swift	2014	4.60	6.87	42450	Diesel	Dealer	Manual	0

Code Block 5: Reading Data

Input

data.tail()

Output

Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner	
296	city	2016	9.50	11.6	33988	Diesel	Dealer	Manual	0
297	brio	2015	4.00	5.9	60000	Petrol	Dealer	Manual	0
298	city	2009	3.35	11.0	87934	Petrol	Dealer	Manual	0

Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner	
299	city	2017	11.50	12.5	9000	Diesel	Dealer	Manual	0
300	brio	2016	5.30	5.9	5464	Petrol	Dealer	Manual	0

Code Block 6: Reading Data

Input

data.describe()

Output

	Year	Selling_Price	Present_Price	Kms_Driven	Owner
count	301.000000	301.000000	301.000000	301.000000	301.000000
mean	2013.627907	4.661296	7.628472	36947.205980	0.043189
std	2.891554	5.082812	8.644115	38886.883882	0.247915
min	2003.000000	0.100000	0.320000	500.000000	0.000000
25%	2012.000000	0.900000	1.200000	15000.000000	0.000000
50%	2014.000000	3.600000	6.400000	32000.000000	0.000000
75%	2016.000000	6.000000	9.900000	48767.000000	0.000000
max	2018.000000	35.000000	92.600000	500000.000000	3.000000

Code Block 7: Reading Data

Input

```
data.info()
```

Output

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 301 entries, 0 to 300
Data columns (total 9 columns):
 #   Column         Non-Null Count  Dtype  
---  ------         --------------  -----  
 0   Car_Name       301 non-null    object 
 1   Year           301 non-null    int64  
 2   Selling_Price  301 non-null    float64
 3   Present_Price  301 non-null    float64
 4   Kms_Driven     301 non-null    int64  
 5   Fuel_Type      301 non-null    object 
 6   Seller_Type    301 non-null    object 
 7   Transmission   301 non-null    object 
 8   Owner          301 non-null    int64  
dtypes: float64(2), int64(3), object(4)
memory usage: 21.3+ KB
```

Code Block 8: Reading Data

Input

```
data.isnull().sum()
```

Output

Car_Name 0

Year 0

Selling_Price 0

Present_Price 0

Kms_Driven 0

Fuel_Type 0

Seller_Type 0

Transmission 0

Owner 0

dtype: int64

Code Block 9: Reading Data

Input

data.shape

Output

(301, 9)

Code Block 10: Reading Data

Input

data['Fuel_Type'].value_counts()

Output

Fuel_Type

Petrol 239

Diesel 60

CNG 2

Name: count, dtype: int64

Code Block 11: Reading Data

Input

data['Seller_Type'].value_counts()

Output

Seller_Type

Dealer 195

Individual 106

Name: count, dtype: int64

Code Block 12: Reading Data

Input

data['Transmission'].value_counts()

Output

Transmission

Manual 261

Automatic 40

Name: count, dtype: int64

Code Block 13: Encoding

Input

data = data.replace({'Fuel_Type':{'Petrol':0, 'Diesel':1, 'CNG':2}})

Output

/tmp/ipykernel_69/3731797036.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer_objects(copy=False)`. To opt-in to the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`

```
data  = data.replace({'Fuel_Type':{'Petrol':0, 'Diesel':1, 'CNG':2}})
```

Code Block 14: Encoding

Input

data

Output

Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner	
0	ritz	2014	3.35	5.59	27000	0	Dealer	Manual	0
1	sx4	2013	4.75	9.54	43000	1	Dealer	Manual	0
2	ciaz	2017	7.25	9.85	6900	0	Dealer	Manual	0
3	wagon r	2011	2.85	4.15	5200	0	Dealer	Manual	0
4	swift	2014	4.60	6.87	42450	1	Dealer	Manual	0
...	...	...	...	...	...	...	...	...	...
296	city	2016	9.50	11.60	33988	1	Dealer	Manual	0

Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner	
297	brio	2015	4.00	5.90	60000	0	Dealer	Manual	0
298	city	2009	3.35	11.00	87934	0	Dealer	Manual	0
299	city	2017	11.50	12.50	9000	1	Dealer	Manual	0
300	brio	2016	5.30	5.90	5464	0	Dealer	Manual	0

301 rows × 9 columns

Code Block 15: Encoding

Input

```
data  = data.replace({'Seller_Type':{'Dealer':0, 'Individual':1}})
```

Output

```
/tmp/ipykernel_69/2879944377.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer_objects(copy=False)`. To opt-in to the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`

  data  = data.replace({'Seller_Type':{'Dealer':0, 'Individual':1}})
```

Code Block 16: Encoding

Input

```
data
```

Output

Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner	
0	ritz	2014	3.35	5.59	27000	0	0	Manual	0
1	sx4	2013	4.75	9.54	43000	1	0	Manual	0
2	ciaz	2017	7.25	9.85	6900	0	0	Manual	0
3	wagon r	2011	2.85	4.15	5200	0	0	Manual	0
4	swift	2014	4.60	6.87	42450	1	0	Manual	0
...	...	...	...	...	...	...	...	...	...
296	city	2016	9.50	11.60	33988	1	0	Manual	0
297	brio	2015	4.00	5.90	60000	0	0	Manual	0
298	city	2009	3.35	11.00	87934	0	0	Manual	0
299	city	2017	11.50	12.50	9000	1	0	Manual	0
300	brio	2016	5.30	5.90	5464	0	0	Manual	0

301 rows × 9 columns

Code Block 17: Encoding

Input

```
data = data.replace({'Transmission':{'Manual':0, 'Automatic':1}})
```

Output

```
/tmp/ipykernel_69/2747165536.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer_objects(copy=False)`. To opt-in to the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`

  data = data.replace({'Transmission':{'Manual':0, 'Automatic':1}})
```

Code Block 18: Encoding

Input

```
data
```

Output

Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner	
0	ritz	2014	3.35	5.59	27000	0	0	0	0
1	sx4	2013	4.75	9.54	43000	1	0	0	0
2	ciaz	2017	7.25	9.85	6900	0	0	0	0
3	wagon r	2011	2.85	4.15	5200	0	0	0	0
4	swift	2014	4.60	6.87	42450	1	0	0	0
...	...	...	...	...	...	...	...	...	...

Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner	
296	city	2016	9.50	11.60	33988	1	0	0	0
297	brio	2015	4.00	5.90	60000	0	0	0	0
298	city	2009	3.35	11.00	87934	0	0	0	0
299	city	2017	11.50	12.50	9000	1	0	0	0
300	brio	2016	5.30	5.90	5464	0	0	0	0

301 rows × 9 columns

Code Block 19: Preprocessing

Input

```
X = data.drop(['Car_Name','Selling_Price'], axis=1)
```

Output

Code Block 20: Preprocessing

Input

```
Y = data['Selling_Price']
```

Output

Code Block 21: Preprocessing

Input

```
print(X.head())
```

Output

```
Year  Present_Price  Kms_Driven  Fuel_Type  Seller_Type  Transmission  \
0  2014         5.59       27000          0            0             0
1  2013         9.54       43000          1            0             0
2  2017         9.85        6900          0            0             0
3  2011         4.15        5200          0            0             0
4  2014         6.87       42450          1            0             0

   Owner
0      0
1      0
2      0
3      0
4      0
```

Code Block 22: Preprocessing

Input

```
Y.head()
```

Output

```
0    3.35
1    4.75
2    7.25
3    2.85
4    4.60
```

Name: Selling_Price, dtype: float64

Code Block 23: Preprocessing (Splitting dataset)

Input

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.15, random_state = 123)

Output

Code Block 24: Model Training

Input

model = LinearRegression()

Output

Code Block 25: Model Training

Input

model.fit(X_train, Y_train)

Output

```
▾ LinearRegression
LinearRegression()
```

Code Block 26: Predicting Outputs

Input

Y_train_pred = model.predict(X_train)

Y_test_pred = model.predict(X_test)

Output

Code Block 27: Model Evaluation

Input

```
mae = mean_absolute_error(Y_train, Y_train_pred)
mse = mean_squared_error(Y_train, Y_train_pred)
rmse = np.sqrt(mse)
r2 = r2_score(Y_train, Y_train_pred)
print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
print("Root MSE (RMSE):", rmse)
print("R² Score:", r2)
```

Output

```
Mean Absolute Error (MAE): 1.1605491562122932
Mean Squared Error (MSE): 3.0156259771916045
Root MSE (RMSE): 1.7365557800403661
R² Score: 0.8869753499147773
```

Code Block 28: Model Evaluation

Input

```
mae = mean_absolute_error(Y_test, Y_test_pred)
mse = mean_squared_error(Y_test, Y_test_pred)
rmse = np.sqrt(mse)
```

```
r2 = r2_score(Y_test, Y_test_pred)

print("Mean Absolute Error (MAE):", mae)

print("Mean Squared Error (MSE):", mse)

print("Root MSE (RMSE):", rmse)

print("R² Score:", r2)
```

Output

```
Mean Absolute Error (MAE): 1.3160017493661493

Mean Squared Error (MSE): 3.8164774430601214

Root MSE (RMSE): 1.9535806722682638

R² Score: 0.8114116982277799
```

Code Block 29: Model Evaluation

Input

```
print("Coefficients:", model.coef_)

print("Intercept:", model.intercept_)
```

Output

```
Coefficients: [ 3.92825483e-01  4.39836584e-01 -5.25747442e-06  1.38880051e+00

 -1.25197058e+00  1.44799328e+00 -7.41998907e-01]

Intercept: -789.4879877877579
```

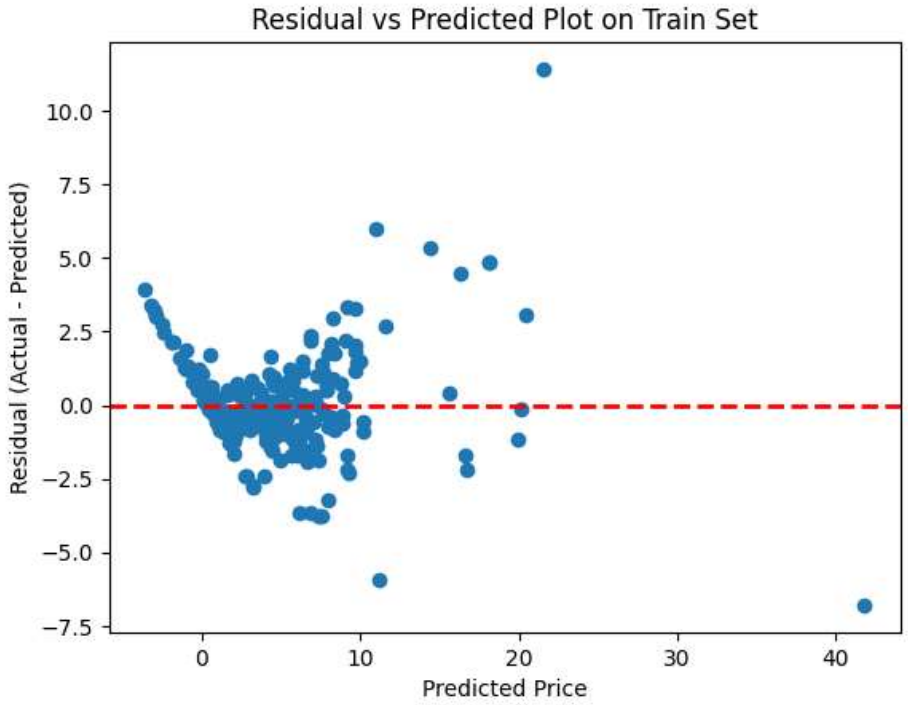
Code Block 30: Model Evaluation

Input

```
residuals = Y_train - Y_train_pred
```

```
plt.scatter(Y_train_pred, residuals)
plt.axhline(0, color='red', linestyle='--', linewidth=2)
plt.xlabel("Predicted Price")
plt.ylabel("Residual (Actual - Predicted)")
plt.title("Residual vs Predicted Plot on Train Set")
plt.show()
```

Output



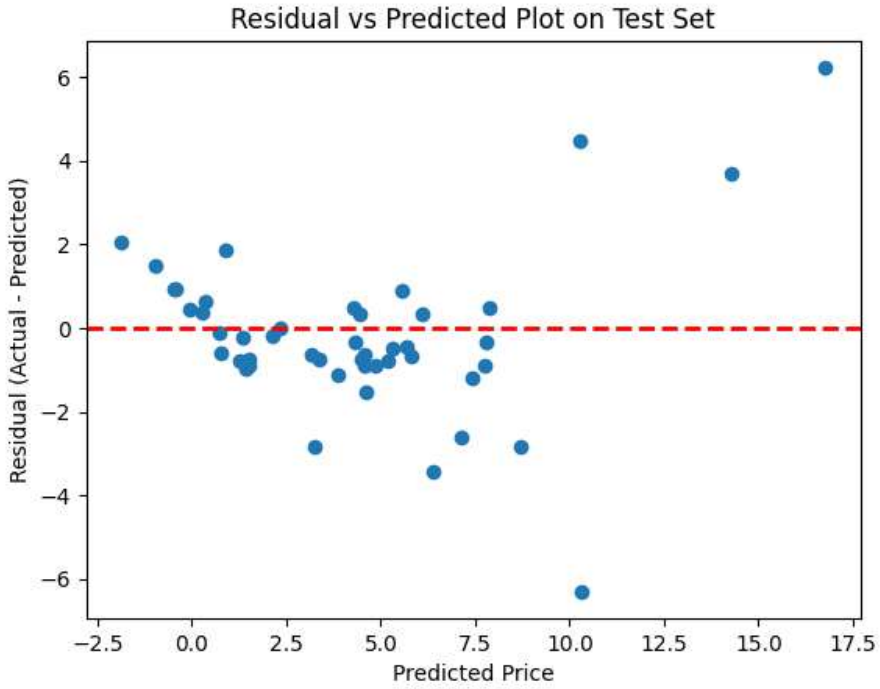
Code Block 31: Visualizations

Input

```
residuals = Y_test - Y_test_pred
plt.scatter(Y_test_pred, residuals)
plt.axhline(0, color='red', linestyle='--', linewidth=2)
```

```
plt.xlabel("Predicted Price")

plt.ylabel("Residual (Actual - Predicted)")

plt.title("Residual vs Predicted Plot on Test Set")

plt.show()
```

Output



Code Block 32: Single Variable Linear Regression; Importing Libraries

Input

```
import pandas as pd

from sklearn.linear_model import LinearRegression
```

Output

Code Block 33: Reading Data

Input

data = pd.read_csv('/kaggle/input/salary-dataset-simple-linear-regression/Salary_dataset.csv')

Output

Code Block 34: Reading Data

Input

data.head()

Output

Unnamed: 0	YearsExperience	Salary	
0	0	1.2	39344.0
1	1	1.4	46206.0
2	2	1.6	37732.0
3	3	2.1	43526.0
4	4	2.3	39892.0

Code Block 35: Reading Data

Input

data.tail()

Output

Unnamed: 0	YearsExperience	Salary	
25	25	9.1	105583.0
26	26	9.6	116970.0
27	27	9.7	112636.0
28	28	10.4	122392.0
29	29	10.6	121873.0

Code Block 36: Reading Data

Input

data.describe()

Output

Unnamed: 0	YearsExperience	Salary	
count	30.000000	30.000000	30.000000
mean	14.500000	5.413333	76004.000000
std	8.803408	2.837888	27414.429785
min	0.000000	1.200000	37732.000000
25%	7.250000	3.300000	56721.750000
50%	14.500000	4.800000	65238.000000
75%	21.750000	7.800000	100545.750000

Unnamed: 0	YearsExperience	Salary	
max	29.000000	10.600000	122392.000000

Code Block 37: Reading Data

Input

```
data.info()
```

Output

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 3 columns):
 #   Column          Non-Null Count  Dtype  
---  ------          --------------  -----  
 0   Unnamed: 0      30 non-null     int64  
 1   YearsExperience  30 non-null     float64
 2   Salary          30 non-null     float64
dtypes: float64(2), int64(1)
memory usage: 852.0 bytes
```

Code Block 38: Reading Data

Input

```
data.isnull().sum()
```

Output

```
Unnamed: 0        0
```

YearsExperience 0

Salary 0

dtype: int64

Code Block 39: Reading Data

Input

data.shape

Output

(30, 3)

Code Block 40: Preprocessing

Input

X = data.drop(['Unnamed: 0','Salary'], axis=1)

Output

Code Block 41: Preprocessing

Input

X.head()

Output

YearsExperience	
0	1.2
1	1.4

YearsExperience	
2	1.6
3	2.1
4	2.3

Code Block 42: Preprocessing

Input

```
Y = data['Salary']
```

Output

Code Block 43: Preprocessing

Input

```
Y.head()
```

Output

```
0    39344.0
1    46206.0
2    37732.0
3    43526.0
4    39892.0
Name: Salary, dtype: float64
```

Code Block 44: Preprocessing (Splitting dataset)

Input

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.15, random_state = 123)

Output

Code Block 45: Model Training

Input

model = LinearRegression()

Output

Code Block 46: Visualizations

Input

model.fit(X_train, Y_train)

Output

```
▾ LinearRegression
LinearRegression()
```

Code Block 47: Model Evaluation

Input

Y_train_pred = model.predict(X_train)

Y_test_pred = model.predict(X_test)

Output

Code Block 48: Model Evaluation

Input

```
mae = mean_absolute_error(Y_train, Y_train_pred)

mse = mean_squared_error(Y_train, Y_train_pred)

rmse = np.sqrt(mse)

r2 = r2_score(Y_train, Y_train_pred)


print("Mean Absolute Error (MAE):", mae)

print("Mean Squared Error (MSE):", mse)

print("Root MSE (RMSE):", rmse)

print("R² Score:", r2)
```

Output

```
Mean Absolute Error (MAE): 4784.217745729295

Mean Squared Error (MSE): 33289045.961436674

Root MSE (RMSE): 5769.666018188286

R² Score: 0.9510236184054837
```

Code Block 49: Model Evaluation

Input

```
mae = mean_absolute_error(Y_test, Y_test_pred)

mse = mean_squared_error(Y_test, Y_test_pred)

rmse = np.sqrt(mse)

r2 = r2_score(Y_test, Y_test_pred)


print("Mean Absolute Error (MAE):", mae)
```

```
print("Mean Squared Error (MSE):", mse)

print("Root MSE (RMSE):", rmse)

print("R² Score:", r2)
```

Output

```
Mean Absolute Error (MAE): 3789.015632367744

Mean Squared Error (MSE): 23401316.80382216

Root MSE (RMSE): 4837.4907549081845

R² Score: 0.9741093885914234
```

Code Block 50: Model Evaluation

Input

```
model.coef_
```

Output

```
array([9635.21736133])
```

Code Block 51: Model Evaluation

Input

```
model.intercept_
```

Output

```
23524.487115498625
```

Code Block 52: Visualization

Input

```
plt.scatter(X_train, Y_train, label='Actual Salary Data')
plt.plot(X_train, Y_train_pred, color='red', label='Fitted Regression Line')
plt.xlabel('Years of Experience'); plt.ylabel('Salary')
plt.title('Linear Regression Fit')
plt.legend(); plt.show()
```

Output

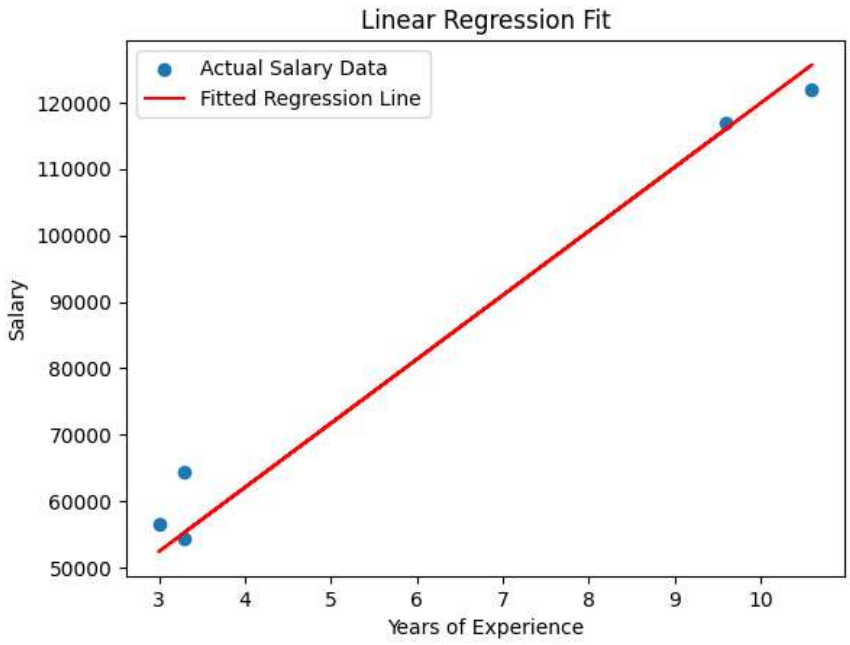
Code Block 53: Visualization

Input

```
plt.scatter(X_test, Y_test, label='Actual Salary Data')
plt.plot(X_test, Y_test_pred, color='red', label='Fitted Regression Line')
plt.xlabel('Years of Experience'); plt.ylabel('Salary')
plt.title('Linear Regression Fit')
```

plt.legend(); plt.show()

Output



Code Block 54: Visualizations

Input

residuals = Y_train - Y_train_pred

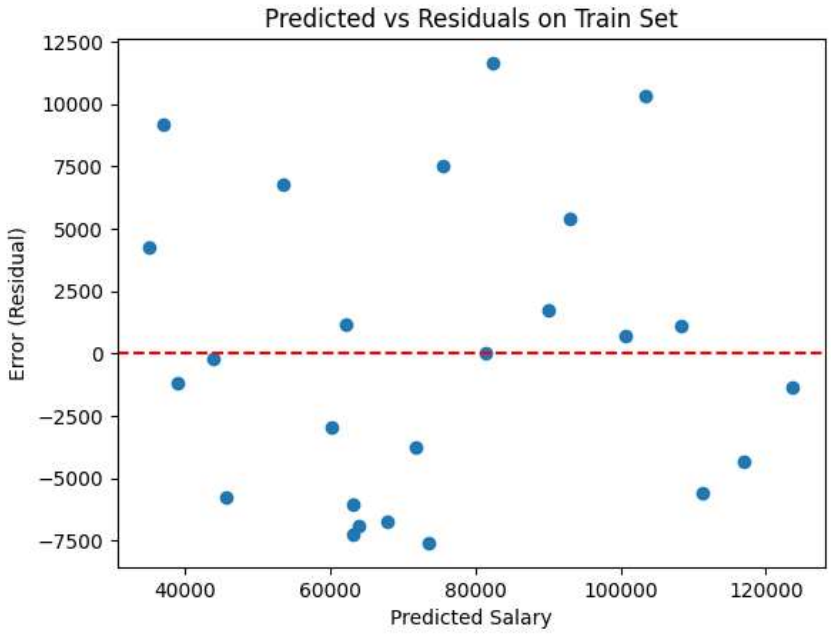
plt.scatter(Y_train_pred, residuals); plt.axhline(y=0, color = 'red', linestyle='--')

plt.xlabel("Predicted Salary")

plt.ylabel("Error (Residual)")

plt.title("Predicted vs Residuals on Train Set"); plt.show()

Output

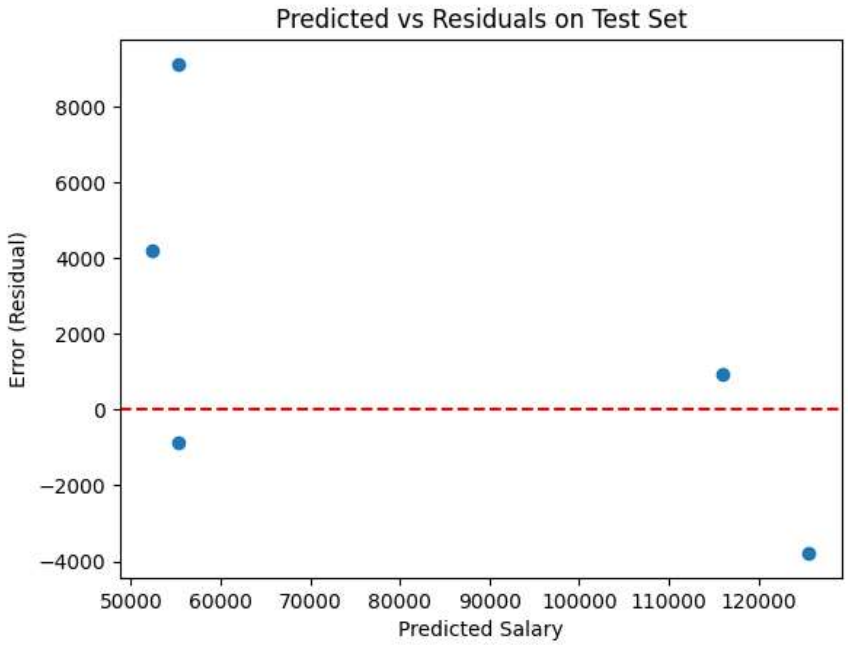


Code Block 55: Visualizations

Input

```
residuals = Y_test - Y_test_pred

plt.scatter(Y_test_pred, residuals)

plt.axhline(y=0, color = 'red', linestyle='--')  # reference line for zero error

plt.xlabel("Predicted Salary")

plt.ylabel("Error (Residual)")

plt.title("Predicted vs Residuals on Test Set")

plt.show()
```

Output



Code Block 56: User Choice Outputs

Input

```
# Predict all test samples
Y_test_pred = model.predict(X_test)


# Create a dataframe for comparison
comparison_df = pd.DataFrame({
    "Actual Selling Price": Y_test,
    "Predicted Selling Price": np.round(Y_test_pred, 2),
    "Error": np.round(Y_test - Y_test_pred, 2),
    "Absolute Error": np.round(abs(Y_test - Y_test_pred), 2)
})


# Sort by smallest error → best predictions
```

```
best_predictions = comparison_df.sort_values(by="Absolute Error").head(5)

print(best_predictions)
```

Output

	Actual Selling Price	Predicted Selling Price	Error	Absolute Error
31	2.35	2.35	0.00	0.00
144	0.60	0.71	-0.11	0.11
42	1.95	2.15	-0.20	0.20
114	1.15	1.36	-0.21	0.21
89	4.75	4.43	0.32	0.32